Java OOP Laboratory Exercise: Campus Café Ordering System

ITS 202: Objected Oriented Programming

Submitted by:

Denzemiel Ethan S. Dacillo

Denemiel Nathan S. Dacillo

Section: C

NOVEMBER 9, 2026

Concept Explanation

if Statement: Used this in Pastry.computePrice() and Meal.computePrice() to add the warming fee or the upsize fee only when it applies. It's also used in Order to skip empty item slots.

if-else Statement: Used when there are two outcomes, such as setBasePrice() rejecting negative prices, Beverage.computePrice() choosing by size, Order.addItem() checking the three-item limit, and computeTotal(double) keeping the total from going below 0.

Classes and Objects: The program has seven classes, each in its own file. CafeApp creates the objects (customers, items and orders) and runs the demo.

Attributes and Behaviors: Variables store an object's data (such as size or warmed), and methods do its actions (such as computePrice() or addItem()).

Object Interaction (Order): An Order holds a Customer and up to three MenuItem objects. printReceipt() uses them and updates the customer's loyalty points.

Java Naming Conventions: Classes use PascalCase (MenuItem) and variables and methods use camelCase (basePrice, computeTotal). The fee amounts are written directly in the formulas, not as constants.

Default Constructor: Sets starting values when no information is given, such as "Unnamed Item" with a price of 0.0 in MenuItem().

Parameterized Constructor: Lets an object get its data when it is created. The subclasses pass the name, price and code up to MenuItem with super(...).

Constructor Overloading: Customer has three constructors, so it can be created with no information, with a name only, or with a name and student status.

public / private / protected: Instance variables are private or protected and methods are public. name and basePrice are protected so the subclasses can use them.

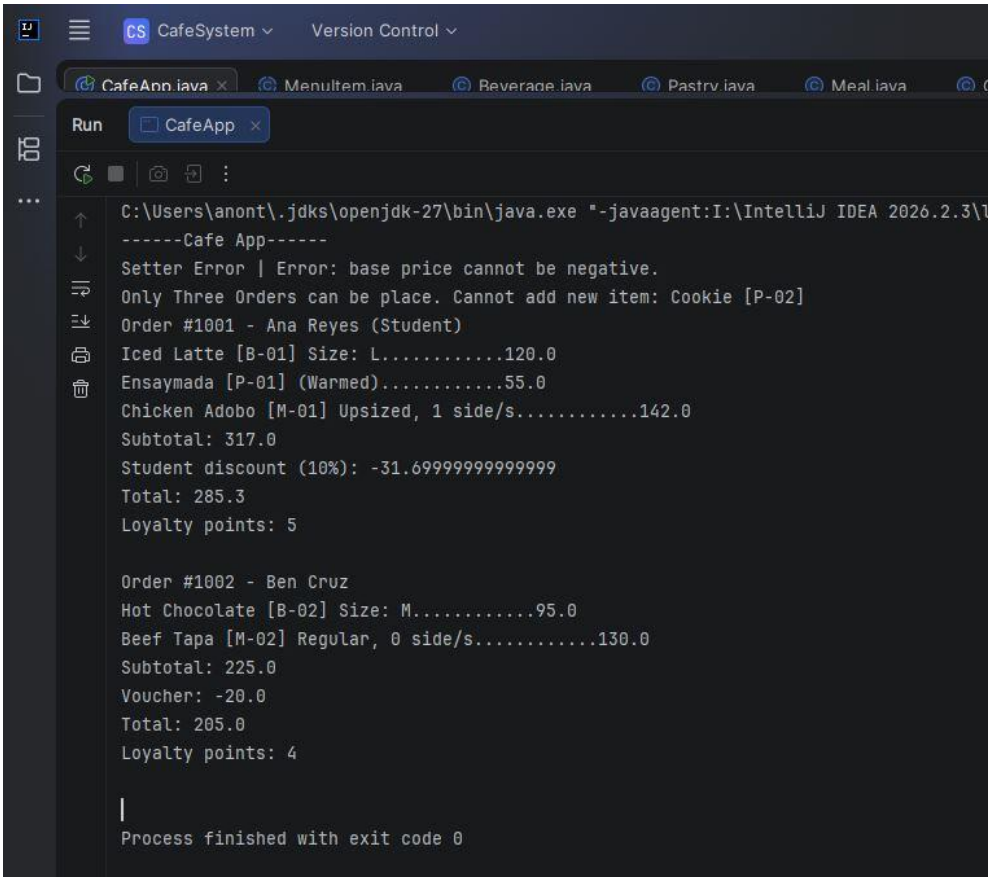
Getters, Setters, Data Hiding: Data is read and changed only through methods, and the setters check the value first (for example, no negative prices or points).

Method Overloading: The same method name with different parameters: addPoints(int) and addPoints(double), computeTotal() and computeTotal(double), and printReceipt() and printReceipt(double).

Method Overriding: Beverage, Pastry and Meal replace computePrice() and getDescription() from MenuItem, using @Override.

Abstraction: MenuItem is abstract and has the abstract method computePrice(), so every subclass must define its own price.

Screenshot / Output:



Challenges Encountered:

- **Comparing Strings.** In Beverage, I compared the size with ==, and in setSize() I joined "S", "M" and "L" into one text, so valid sizes were rejected. I fixed it by comparing each size with .equals() and joining the three checks with ||.
- **Size left empty.** When the Beverage constructor received an invalid size, the setter rejected it and size stayed null, which would crash computePrice(). I solved it by setting size to "S" before calling setSize().
- **Abstract method.** At first MenuItem was not declared abstract, and I gave computePrice() a body. I fixed it by adding the abstract keyword to the class and ending the method with a semicolon.
- **Points in Customer.** My addPoints() methods replaced the points instead of adding to them, and my negative checks were backwards, so valid values gave errors. I fixed them by checking the incoming value (points and amountSpent) and using +=, and I used a cast to turn the amount spent into whole points (1 per 50 pesos).
- **Using += and *= correctly.** In Order, I wrote the operation on a method call (such as item1.computePrice() += subTotal), which would not compile, and I returned the wrong value at the end of two methods. I fixed these by using a variable for the running subtotal and total, and returning that variable.